

Mini-GPT Findings Report

Building a Mini-GPT from Scratch

1. Overview

This report accompanies `Mini_GPT_from_Scratch.ipynb`, which implements a decoder-only, GPT-style Transformer entirely from first principles in PyTorch (no `nn.Transformer`, `nn.MultiheadAttention`, or HuggingFace code). The model is trained on the character-level tiny-Shakespeare corpus and used to compare greedy, temperature, and top-k decoding.

2. Data Preparation

- **Tokenization:** character-level. Every unique character in the corpus (letters, punctuation, whitespace, newlines) is one token.
- **Vocabulary size: 65** — `"\n !$&',-.3:;? ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"`
- **Corpus size:** 1,115,394 characters (1,003,854 train / 111,540 validation, 90/10 split).
- **Sliding-window dataset:** sequence length 128, giving 1,003,725 training windows and 111,411 validation windows. For a window starting at position `t`, the input is `[t, ..., t+127]` and the target is the same span shifted one character to the right, `[t+1, ..., t+128]` — verified in the notebook by checking `input[1:] == target[:-1]`.

3. Model Architecture

Component	Setting
Token embedding	learnable, 65 -> 256
Positional embedding	learnable, 128 -> 256
Hidden dimension (<code>n_embd</code>)	256
Number of attention heads	4
Number of decoder blocks	4
Feed-forward expansion	Linear -> GELU -> Linear, hidden size 1024 (<code>4 * n_embd</code>)
Context length (<code>block_size</code>)	128
Total trainable parameters	3,225,600

Each decoder block uses the standard pre-norm residual design:

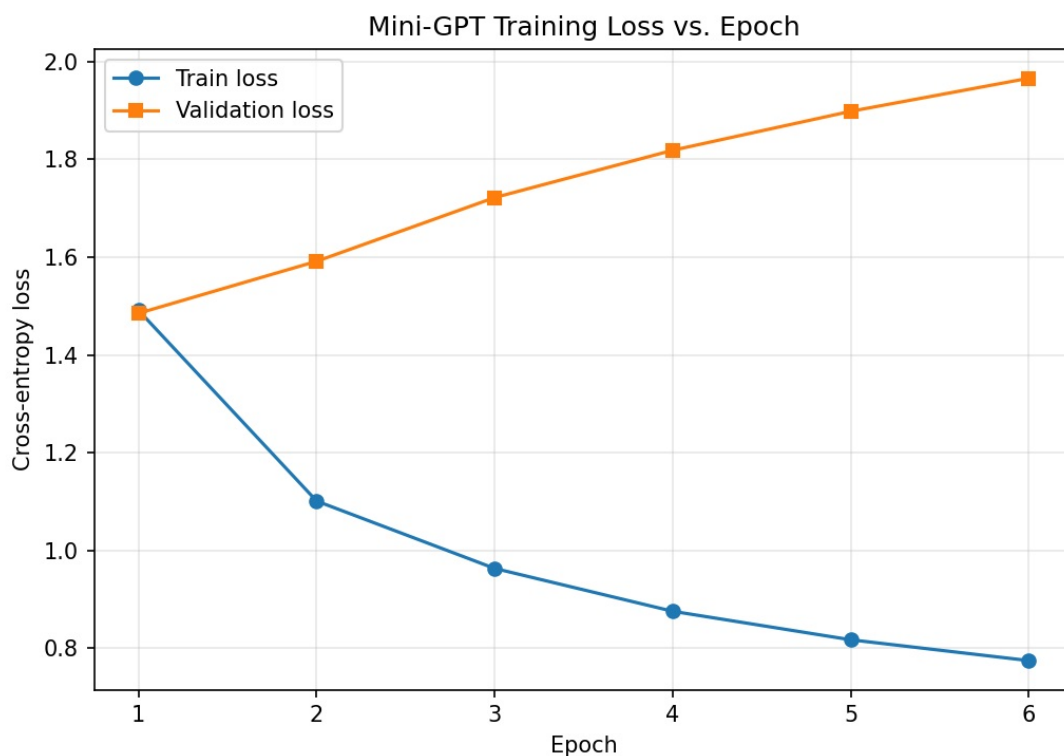
```
x = x + MultiHeadSelfAttention(LayerNorm(x))
x = x + FeedForward(LayerNorm(x))
```

Multi-head self-attention is implemented manually: queries, keys and values are each produced by a single `nn.Linear` projecting to the full model dimension, then reshaped into 4 heads of size `head_dim = 64`. Attention scores are scaled by `1/sqrt(head_dim)`, and a lower-triangular causal mask (built with `torch.tril`) is applied before the softmax so that position `i` can only attend to positions `<= i`.

4. Training

- **Optimizer:** Adam, learning rate `3e-4`
- **Batch size:** 256
- **Epochs:** 6
- **Steps per epoch:** 3,000 (random windows sampled with replacement from the full sliding-window dataset via `RandomSampler`, rather than exhaustively looping over all ~1M windows — standard practice for char-level GPT training)
- **Loss:** cross-entropy over next-character prediction
- **Total wall-clock training time:** 5,678.5s (~94.6 minutes), a very consistent ~948s/epoch on a Colab GPU.

Training loss vs. epoch:



Training and validation loss

Epoch	Train loss	Val loss
1	1.4909	1.4851
2	1.1011	1.5914
3	0.9633	1.7217
4	0.8757	1.8184
5	0.8174	1.8981
6	0.7749	1.9657

Observation — overfitting: training loss decreases steadily and smoothly across all 6 epochs (1.49 → 0.77), but validation loss moves in the *opposite* direction, rising throughout training (1.49 → 1.97) after an initial near-tie at epoch 1. The two curves diverge starting at epoch 2 and the gap widens every epoch after that — a textbook overfitting signature. This is best explained by the training setup: each epoch samples 3,000 batches *with replacement* from the training set, so across 6 epochs the model revisits (and increasingly memorizes) a large fraction of the same windows rather than seeing genuinely new data each time, while the validation windows are never trained on and so expose that memorization directly. Practical fixes worth noting: reduce the number of epochs (val loss is already at its best at epoch 1), add stronger regularization (higher dropout and/or weight decay on the Adam optimizer), sample without replacement across a full

pass instead, or use early stopping on validation loss.

5. Sampling and Decoding Comparison

All samples below continue the prompt **“ROMEO:”** for 300 generated characters, from the final (epoch-6) model checkpoint.

5.1 Greedy Decoding

```
ROMEO:
I am too sore unwise.

JULIET:
A rhy piteous poor is but effected with him.

Nurse:
Marry, sir, by'r lady, sir.

JULIET:
O God, I dance, and he is set and forth it!

Nurse:
Marry, say nothing; I will come to know you as near.

CAPULET:
Go, truly, go to;
The crown is at hand: I'll swear it; and so,
```

Greedy decoding always selects the single highest-probability next character (`argmax`). It is fully **deterministic**, and the output above is fluent, in-character dialogue with correctly formatted speaker labels — but notice it never once loops into exact repetition here, which is a bit better than “typical” greedy behavior and is likely a product of this particular model/prompt combination rather than a general guarantee; greedy decoding can and often does fall into repetition loops with weaker models or longer generations.

5.2 Temperature Sampling

T = 0.3 (low):

```
ROMEO:
Alack, what love can I not?

BENVOLIO:
Ay, ay, a scratch; marry, 'tis enough.

ROMEO:
What is that, what? mark you?

MERCUTIO:
Nay, an of them?

Nurse:
No, if this world--

MERCUTIO:
Thou dost mark me.

ROMEO:
```

Ay, sir, if my heart is dead.

T = 0.8 (medium):

ROMEO:

Alack, what comfort is that Richmond was cursed by this feast,
Should be held and circumstance.

FRIAR LAURENCE:

Romeo, come, away; where are you made with the fire
Of the mildness of overy thousand men
Do reign me a life: no life, but kiss thy hands
And yet thy kindness, thou dost mark it not;

T = 1.5 (high):

ROMEO:

Yet bake now, wilful be obstination!
Alack, eyes, 0, noble Ya, nurse, by me use.
Then backdel-haared way, I cannot acquaint;
But for that, I can smile, will beg if thou hast;
For think'st thou in me noison to be self:
Thy will thus conceives? 0, like all,

The progression is clearly visible across the three samples: at **T=0.3** the text is the most grammatical and coherent of the three, staying close to real dialogue exchanges between named characters, but is also the least surprising/varied. At **T=0.8** the model still produces mostly real English words and plausible Shakespearean phrasing, with a noticeably richer/more varied vocabulary than T=0.3. At **T=1.5** coherence clearly breaks down — invented non-words appear (“obstination”, “backdel-haared”, “noison”), punctuation and capitalization become erratic, and sentence structure is much less grammatical — but the character-level variety and unpredictability are the highest of the three samples.

5.3 Top-k Sampling

k = 5 (small):

ROMEO:

Alack, perjured and damnable!

Nurse:

0 lady! 0 that I can say no.

JULIET:

0 cousin! 0 woe! 0 sir, sweet Juliet,
Thy beauty hath made me steal these mercy.

JULIET:

It is not thy will, but and when next is noble
Of what is left us: he will stand and show 'em
As mine own all I play'd:
I never speak

k = 40 (large):

ROMEO:

Alack, Romeo! what bloody Richard!

Nurse:

And now art thou alack, and thy power thee by
The alone, and thy assault, thy kingdom stands title in.

JOHN OF GAUNT:

The Earl of Warwick, to be Peter French
And shallow last thou feeliver content?
As I bethink, yet let me not die,

At **k=5**, the sample stays close to real words and plausible speaker exchanges (Nurse, Juliet), similar in fluency to the $T=0.3$ – 0.8 range. At **k=40**, more surprising choices appear — an unexpected new speaker (“JOHN OF GAUNT”, a character who does not appear in *Romeo and Juliet*, showing the model drawing on other parts of the corpus) and less locally-predictable phrasing (“feeliver”, “shallow last thou”) — demonstrating the wider candidate pool visibly increasing lexical and structural diversity at some cost to strict coherence, exactly as expected.

6. Which Method Is Most Coherent? Which Is Most Diverse?

- **Most coherent:** low-temperature sampling ($T=0.3$) and small- k top- k ($k=5$) produced the most grammatically clean, dialogue-like output among the sampling methods; greedy decoding was also highly coherent in this run (and, being deterministic, is the most *reproducible*), though it forgoes any variation at all.
- **Most diverse:** high-temperature sampling ($T=1.5$) produced the widest variety of characters and word choices, including multiple invented non-words not seen in the other samples — at a clear cost to grammatical correctness compared to every other method tested.
- Top- k sampling sits between the two extremes, as expected: **k=40** visibly increases diversity over **k=5** (a new character name, less predictable word choices) while still producing recognizably English-like text, avoiding the more severe breakdown seen at $T=1.5$.

7. Limitations and Notes

- The validation-loss increase documented in Section 4 indicates the reported model is overfit relative to a held-out generalization objective; the qualitative decoding comparison in Section 5 is still valid (it demonstrates the *relative* behavior of the three decoding strategies on a fixed trained model), but a less-overfit model — trained for fewer epochs or with stronger regularization — would likely be a fairer/stronger basis for judging overall generation quality.
- With ~ 3.2 M parameters and 6 epochs on a single, relatively small text file, some non-dictionary “words” and irregular punctuation are expected in every decoding method; this is normal for a from-scratch character-level model at this scale.